

Submission: hw03

Zidane Karim <zidane.karim@cooper.edu>

10 Source Files

0 Plots

536 Lines

Generated: 2026-09-23 20:33:05

Source Code & Configuration

pyproject.toml

33 lines

```
1  # pyproject.toml.jinja
2
3  [project]
4  name = "hw03"
5  version = "0.1.0"
6  description = "Convolutional Neural Network Classifier"
7  readme = "README.md"
8  authors = [
9      { name = "Zidane Karim", email = "zidane.karim@cooper.edu" }
10 ]
11 requires-python = ">=3.13"
12 dependencies = [
13     "structlog",
14     "rich",
15     "numpy",
16     "pydantic",
17     "pydantic-settings>=2.10.1",
18     "matplotlib>=3.10.5",
19     "tqdm>=4.67.1",
20     "jax[cpu,cuda13]==0.11.1",
21     "flax>=0.11.2",
22     "optax>=0.2.4",
23     "scikit-learn",
24     "jaxlib[cuda13]==0.11.1",
25     "datasets>=5.0.1",
26 ]
27
28 [project.scripts]
29 hw03 = "hw03:main"
30 hw03-evaluate = "hw03.evaluate:main"
31 [build-system]
32 requires = ["uv_build>=0.8.3,<0.9.0"]
33 build-backend = "uv_build"
```

README.md

Markdown

5 lines

hw03

Convolutional Neural Network Classifier

Test Accuracy: 96.74%

src/hw03/config.toml

17 lines

```
1  debug = false
2  random_seed = 0xc61dd8fc
3
4  [data]
5  num_features = 1
6  num_samples = 50
7  sigma_noise = 0.1
8
9  [training]
10 batch_size = 64
11 num_iters = 4500 # less than 5 epochs
12 learning_rate = 0.001
13
14 [plotting]
15 output_dir = "artifacts"
16 figsize = [5, 3]
17 dpi = 200
```

src/hw03/__init__.py

56 lines

```

1  import jax
2  import numpy as np
3  import optax
4  import structlog
5  from flax import nnx
6
7  from .config import load_settings
8  from .data import Data
9  from .logging import configure_logging
10 from .model import LinearModel, NNXLinearModel, Classifier
11
12 # from .plotting import compare_linear_models, plot_fit
13 from .training import train, evaluate
14
15
16 def main() -> None:
17     """CLI entry point."""
18     settings = load_settings()
19     configure_logging()
20     log = structlog.get_logger()
21     log.info("Settings loaded", settings=settings.model_dump())
22
23     # JAX PRNG
24     key = jax.random.key(settings.random_seed)
25     data_key, model_key = jax.random.split(key)
26     np_rng = np.random.default_rng(np.asarray(jax.random.key_data(data_key)))
27
28     # log.debug("Data generating model", model=data_generating_model)
29
30     data = Data(batch_size=settings.training.batch_size)
31
32     model = Classifier(
33         input_channels=1, # mnist specs
34         layer_channels=[32, 64, 128],
35         kernel_sizes=[(3, 3), (3, 3), (3, 3)],
36         strides=[1, 2, 2], # originally [1,1], which produced 91.2%
37         num_classes=10, # digits 0-9
38         rngs=nnx.Rngs(params=model_key),
39     )
40
41     optimizer = nnx.Optimizer(
42         model, optax.adam(settings.training.learning_rate), wrt=nnx.Param
43     )
44
45     train(model, optimizer, data, settings.training, np_rng)
46
47     # log.debug("Trained model", model=model.model)
48
49     # compare_linear_models(data.model, model.model)
50
51     # if settings.data.num_features == 1:
52     #     plot_fit(model, data, settings.plotting)
53     # else:
54     #     log.info("Skipping plotting for multi-feature models.")
55     test_accuracy = evaluate(model, data.validation_data, batch_size=250)
56     print(f"\nTest Accuracy: {test_accuracy * 100}%\n")

```

src/hw03/config.py

98 lines

```
1  from pathlib import Path
2  from importlib.resources import files
3  from typing import Tuple
4
5  from pydantic import BaseModel, Field
6  from pydantic_settings import (
7      BaseSettings,
8      PydanticBaseSettingsSource,
9      SettingsConfigDict,
10     TomlConfigSettingsSource,
11 )
12
13
14 def _find_config_toml() -> Path:
15     """Resolve config.toml location via importlib.resources or relative path fallback."""
16     if __package__:
17         try:
18             return Path(str(files(__package__).joinpath("config.toml")))
19         except Exception:
20             pass
21     return Path(__file__).parent / "config.toml"
22
23
24 class DataSettings(BaseModel):
25     """Settings for data generation."""
26
27     num_features: int = Field(
28         default=1, gt=0, description="Number of feature dimensions"
29     )
30     num_samples: int = Field(
31         default=50, gt=0, description="Number of synthetic data points"
32     )
33     sigma_noise: float = Field(
34         default=0.5, ge=0.0, description="Standard deviation of Gaussian noise"
35     )
36
37
38 class TrainingSettings(BaseModel):
39     """Settings for model training."""
40
41     batch_size: int = Field(default=16, gt=0, description="Mini-batch size for SGD")
42     num_iters: int = Field(
43         default=300, gt=0, description="Number of optimization iterations"
44     )
45     learning_rate: float = Field(
46         default=0.1, gt=0.0, description="Learning rate for Adam optimizer"
47     )
48
49
50 class PlottingSettings(BaseModel):
51     """Settings for plotting."""
52
53     figsize: Tuple[int, int] = (5, 3)
54     dpi: int = Field(default=200, gt=0, description="Dots per inch for saved figures")
55     output_dir: Path = Path("artifacts")
56
```

```
57
58 class AppSettings(BaseSettings):
59     """Main application settings."""
60
61     debug: bool = False
62     random_seed: int = 31415
63     data: DataSettings = DataSettings()
64     training: TrainingSettings = TrainingSettings()
65     plotting: PlottingSettings = PlottingSettings()
66
67     model_config = SettingsConfigDict(
68         toml_file=_find_config_toml(),
69         env_nested_delimiter="__",
70         extra="ignore",
71     )
72
73     @classmethod
74     def settings_customise_sources(
75         cls,
76         settings_cls: type[BaseSettings],
77         init_settings: PydanticBaseSettingsSource,
78         env_settings: PydanticBaseSettingsSource,
79         dotenv_settings: PydanticBaseSettingsSource,
80         file_secret_settings: PydanticBaseSettingsSource,
81     ) -> tuple[PydanticBaseSettingsSource, ...]:
82         """
83         Set the priority of settings sources.
84
85         We use a TOML file for configuration.
86         """
87         return (
88             init_settings,
89             TomlConfigSettingsSource(settings_cls),
90             env_settings,
91             dotenv_settings,
92             file_secret_settings,
93         )
94
95
96 def load_settings() -> AppSettings:
97     """Load application settings."""
98     return AppSettings()
```

src/hw03/data.py

59 lines

```

1  from dataclasses import InitVar, dataclass, field
2  from datasets import load_dataset
3  import numpy as np
4
5
6  def preprocess(batch):
7      images = [np.array(img, dtype=np.float32) for img in batch["image"]]
8
9      images = np.stack(images, axis=0)
10     images = np.expand_dims(
11         images, axis=-1
12     ) # add channel dimension for grayscale images, since our flax conv wants format: batch,
height, width, channels
13     images = (
14         images / 255.0
15     ) # we normalize bc the pixel values are between 0 and 255, and we want them to be between
0 and 1. this is usual for image data when taking from huggingface or similar
16
17     labels = np.array(batch["label"], dtype=np.int32)
18     return {"image": images, "label": labels}
19
20
21 def load_mnist_dataset():
22     dataset = load_dataset("ylecun/mnist")
23     train_data = dataset["train"].map(preprocess, batched=True)
24     validation_data = dataset["test"].map(preprocess, batched=True)
25
26     return train_data, validation_data
27
28
29 @dataclass
30 class Data:
31     """Handles generation of synthetic data for linear regression."""
32
33     batch_size: int
34
35     def __post_init__(self):
36         self.train_data, self.validation_data = load_mnist_dataset()
37         self.train_data.set_format(
38             type="numpy", columns=["image", "label"]
39         ) # numpy technique to speed computation, nothing crazy
40         self.validation_data.set_format(
41             type="numpy", columns=["image", "label"]
42         ) # numpy technique to speed computation, nothing crazy
43
44         # self.train_images = self.train_data["image"]
45         # self.train_labels = self.train_data["label"]
46         batch = self.train_data[:]
47         self.train_images = np.asarray(batch["image"], dtype=np.float32)
48         self.train_labels = np.asarray(batch["label"], dtype=np.int32)
49
50         self.num_train_samples = len(self.train_data)
51         self.index = np.arange(self.num_train_samples)
52
53     def get_batch(
54         self, rng: np.random.Generator, batch_size: int

```

```
55     ) -> tuple[np.ndarray, np.ndarray]:
56         """Select random subset of examples for training batch."""
57         choices = rng.choice(self.index, size=batch_size, replace=False)
58         batch = self.train_data[choices]
59         return batch["image"], batch["label"]
```

src/hw03/model.py

111 lines

```
1  from dataclasses import dataclass
2
3  import jax
4  import jax.numpy as jnp
5  import numpy as np
6  from flax import nnx
7
8
9  @dataclass
10 class LinearModel:
11     """Represents a simple linear model."""
12
13     weights: np.ndarray
14     bias: float
15
16
17 class Conv2d(nnx.Module):
18     def __init__(
19         self,
20         in_features,
21         out_features,
22         kernel_size,
23         strides,
24         padding,
25         use_bias,
26         *,
27         rngs,
28     ):
29         self.conv = nnx.Conv(
30             in_features=in_features,
31             out_features=out_features,
32             kernel_size=kernel_size,
33             strides=strides,
34             padding=padding,
35             use_bias=use_bias,
36             rngs=rngs,
37         )
38
39     def __call__(self, x: jnp.ndarray) -> jnp.ndarray:
40         return self.conv(x)
41
42
43 class Classifier(nnx.Module):
44     def __init__(
45         self,
46         input_channels: int,
47         layer_channels: list[int],
48         kernel_sizes: list[tuple[int, int]],
49         strides: list[int],
50         num_classes: int,
51         *,
52         rngs,
53     ):
54         self.conv_layers = nnx.List([])
55         self.input_channels = input_channels
56         self.linear_layer = NNLinearModel(
```

```

57         rngs, in_features=layer_channels[-1], out_features=num_classes
58     )
59
60     for output_channel, kernel_size, stride in zip(
61         layer_channels, kernel_sizes, strides
62     ):
63         self.conv_layers.append(
64             Conv2d(
65                 in_features=self.input_channels,
66                 out_features=output_channel,
67                 kernel_size=kernel_size,
68                 strides=stride,
69                 padding="SAME",
70                 use_bias=True,
71                 rngs=rngs,
72             ) # padding="SAME" to keep the output size the same as input size
73         )
74         self.input_channels = output_channel # similar to resnet where we set the input
channels to the output channels of the previous layer
75
76     def __call__(self, x: jnp.ndarray) -> jnp.ndarray:
77         for conv_layer in self.conv_layers:
78             x = nnx.relu(
79                 conv_layer(x)
80             ) # convolutional layers commonly use ReLU as the activation function
81         x = jnp.mean(
82             x, axis=(1, 2)
83         ) # global average pooling so model is invariant to input resolution
84         x = self.linear_layer(x)
85         return x
86
87
88     class NNLinearModel(nnx.Module):
89         """A Flax NN module for a linear regression model, with multi-dimension support added to
solve MLP"""
90
91         def __init__(self, rngs: nnx.Rngs, in_features: int, out_features: int):
92             key = rngs.params()
93             self.in_features = in_features # bc of MLP
94             self.out_features = out_features
95             std = jnp.sqrt(
96                 2.0 / in_features
97             ) # prevents vanishing gradients, switched to He initialization
98             self.w = nnx.Param(jax.random.normal(key, (in_features, out_features)) * std)
99             self.b = nnx.Param(jnp.zeros((1, out_features))) # broadcasts for output
100
101         def __call__(self, x: jax.Array) -> jax.Array:
102             """Predicts the output for a given input."""
103             return x @ self.w.value + self.b.value # squeeze gone to maintain hidden layers
104
105         # @property
106         # def model(self) -> LinearModel:
107         #     """Returns the underlying simple linear model."""
108         #     return LinearModel(
109         #         weights=np.array(self.w.value).reshape([self.num_features]),
110         #         bias=float(np.array(self.b.value).squeeze()),
111         #     )

```

src/hw03/training.py

80 lines

```

1  import jax.numpy as jnp
2  import numpy as np
3  import structlog
4  from flax import nnx
5  from tqdm import trange
6  import optax
7  from .config import TrainingSettings
8  from .data import Data
9  from .model import NNLinearModel
10
11  log = structlog.get_logger()
12
13
14  @nnx.jit
15  def train_step(model, optimizer: nnx.Optimizer, x: jnp.ndarray, y: jnp.ndarray):
16      """Performs a single training step."""
17
18      def loss_fn(model):
19          logits = model(x)
20          return jnp.mean(optax.softmax_cross_entropy_with_integer_labels(logits, y))
21
22      loss, grads = nnx.value_and_grad(loss_fn)(model)
23      optimizer.update(
24          model, grads
25      ) # In-place update of model parameters, same as second assignment
26      return loss
27
28
29  def train(
30      model,
31      optimizer: nnx.Optimizer,
32      data: Data,
33      settings: TrainingSettings,
34      np_rng: np.random.Generator,
35  ) -> None:
36      log.info("Starting training", **settings.model_dump())
37      bar = trange(settings.num_iters)
38      for i in bar:
39          x_np, y_np = data.get_batch(np_rng, settings.batch_size)
40          x, y = jnp.asarray(x_np), jnp.asarray(y_np)
41
42          loss = train_step(model, optimizer, x, y)
43          bar.set_description(f"Loss @ {i} => {loss:.6f}")
44          bar.refresh()
45      log.info("Training finished")
46
47
48      """Evaluation logic to get test accuracy"""
49      # just copy the training logic but against validation
50
51
52  @nnx.jit
53  def eval_step(model: nnx.Module, x: jnp.ndarray, y: jnp.ndarray):
54      logits = model(x)
55      predicted_classes = jnp.argmax(logits, axis=-1)
56      correct_count = jnp.sum(predicted_classes == y)

```

```
57     return correct_count, x.shape[0]
58
59
60 def evaluate(model: nn.Module, test_dataset, batch_size: int = 256) -> float:
61     total_correct = 0
62     total_samples = 0
63
64     num_samples = len(test_dataset)
65     for start_idx in range(0, num_samples, batch_size):
66         end_idx = min(start_idx + batch_size, num_samples)
67         batch = test_dataset[start_idx:end_idx]
68
69         x = jnp.asarray(batch["image"], dtype=jnp.float32)
70         if x.ndim == 3:
71             x = jnp.expand_dims(x, axis=-1)
72
73         y = jnp.asarray(batch["label"])
74
75         correct, count = eval_step(model, x, y)
76         total_correct += int(correct)
77         total_samples += int(count)
78
79     accuracy = total_correct / total_samples
80     return accuracy
```

src/hw03/evaluate.py

14 lines

```
1  """Evaluation entry point for hw03."""
2
3  import structlog
4
5  from .logging import configure_logging
6
7
8  def main() -> None:
9      """CLI entry point for homework evaluation."""
10     configure_logging()
11     log = structlog.get_logger()
12     log.info(
13         "Evaluation not yet implemented for hw03. Implement evaluation logic in src/hw03/
evaluate.py"
14     )
```

src/hw03/logging.py

63 lines

```
1  import logging
2  import os
3  import sys
4  from pathlib import Path
5
6  import jax
7  import numpy as np
8  import structlog
9
10
11  class FormattedFloat(float):
12      def __repr__(self) -> str:
13          return f"{self:.4g}"
14
15
16  def custom_serializer_processor(logger, method_name, event_dict):
17      for key, value in event_dict.items():
18          # Handle JAX arrays in addition to TF tensors
19          if hasattr(value, "numpy"): # Covers TF tensors
20              value = value.numpy()
21          if isinstance(value, jax.Array):
22              value = np.array(value)
23          if isinstance(value, (np.generic, np.ndarray)):
24              value = value.item() if value.size == 1 else value.tolist()
25          if isinstance(value, float):
26              value = FormattedFloat(value)
27          if isinstance(value, Path):
28              value = str(value)
29          event_dict[key] = value
30      return event_dict
31
32
33  def configure_logging():
34      """Configure logging for the application."""
35      log_level = os.environ.get("LOG_LEVEL", "INFO").upper()
36
37      logging.basicConfig(
38          format="%(message)s",
39          stream=sys.stdout,
40          level=log_level,
41          force=True,
42      )
43      logging.getLogger().setLevel(log_level)
44      structlog.configure(
45          processors=[
46              structlog.stdlib.filter_by_level,
47              structlog.stdlib.add_logger_name,
48              structlog.stdlib.add_log_level,
49              structlog.stdlib.PositionalArgumentsFormatter(),
50              structlog.processors.TimeStamper(fmt="iso"),
51              structlog.processors.StackInfoRenderer(),
52              structlog.processors.format_exc_info,
53              structlog.processors.UnicodeDecoder(),
54              custom_serializer_processor,
55              structlog.dev.ConsoleRenderer(
56                  colors=True, exception_formatter=structlog.dev.RichTracebackFormatter()
```

```
57         ),
58     ],
59     context_class=dict,
60     logger_factory=structlog.stdlib.LoggerFactory(),
61     wrapper_class=structlog.stdlib.BoundLogger,
62     cache_logger_on_first_use=True,
63 )
```